Tutorial: Assembly Structure with `as1-oc-214.stp`

Where the OCC Bottle tutorial builds a single part from nothing, this one is about KodaCAD's handling of an assembly that already exists. `as1-oc-214.stp` is a genuine multi-part assembly (a top assembly `as1`, with a pair of shared L-bracket assemblies among its components) -- the natural file for exercising how KodaCAD loads, displays, and modifies structure it didn't just build itself.

This tutorial covers only the material specific to *this* file. Sibling- assembly creation, reparenting, creating and positioning a shared instance from scratch, and undo/redo across a mixed chain of operations are covered instead by the Quaoar Chassis tutorial (../chassis/README.md), which builds that structure directly rather than exploring a pre-built one -- a more concrete way to exercise the same mechanisms.

Why `/` is guaranteed, and why that's worth knowing

KodaCAD has no native save format of its own -- every session is loaded and saved through STEP. Because of that, KodaCAD holds to one strict guarantee: once a document exists at all, `/` is always the top assembly, full stop, through any number of save/reload cycles. Everything else in this tutorial -- and in KodaCAD generally -- depends on that being reliably true.

That guarantee is upheld from both directions. On save, KodaCAD never writes a file with `/` stripped or omitted, for any reason -- so a file KodaCAD produces always has exactly one `/` at its own root. On import, `/` wrappers in an *incoming* file are unwrapped first, recursively however many levels deep, before that file's real content is nested under the current session's own `/` -- so bringing in another KodaCAD-produced file, which always carries its own `/`, never creates `/` nested inside `/`. Step 1 below exercises both directions directly.

The same underlying choice explains why load/save and undo/redo both tend to behave predictably: KodaCAD adheres strictly to OCC's own XDE document model (XCAF) rather than maintaining a parallel, custom representation of a session's own structure. STEP round-tripping and undo/redo both come largely for free as a result -- STEPCAFControl's own reader and writer are built specifically to round-trip an XCAF document through STEP, and KodaCAD's own undo/redo is a thin wrapper around OCAF's own native document-transaction commands, not a custom undo stack built alongside it. Working with the document model rather than around it is what makes both of these dependable rather than fragile.

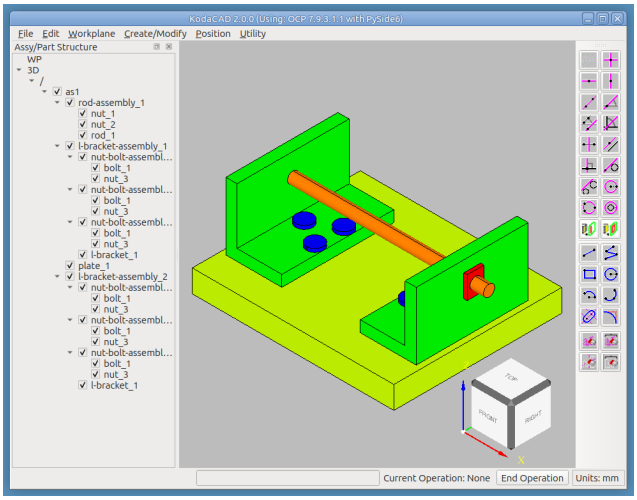
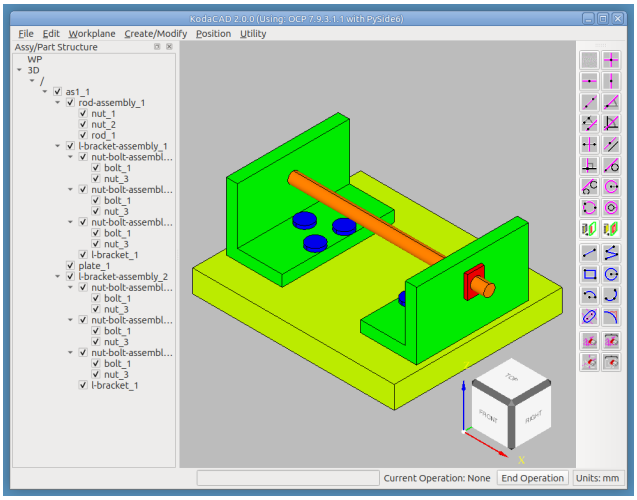
What this exercises

Why Load Session and Import STEP differ, and which to reach for; the XDE hierarchy viewer; recognizing a shared prototype from its referring components; fillet propagation across two *pre-existing* shared instances (as opposed to the Chassis tutorial's freshly-created one); toggling transparency from the tree's own RMB menu; deleting one of two shared instances and confirming the other survives intact; and Pull's Remove Material operation, through the dialog's own operation-linked direction default.

Step 1 -- Load the file two ways: replace, or add

1. **File -> Load Session**, choose `as1-oc-214.stp`. The assembly appears under `/` -- this *replaces* whatever was open.
2. Restart, then **File -> Import STEP**, choose the same file. This time it's *added* as a new component under the current `/`, alongside anything else already open.

Use **Load Session** to pick up exactly where a previous session left off -- nothing else in the current session is preserved. Use **Import STEP** to bring a file's content into a session you're actively building -- everything already there stays, and the import lands alongside it.

Load file `as1-oc-214.stp` as session	Import file `as1-oc-214.stp` into empty session
	

Exercises: the two loading mechanisms documented in the README's "Loading a STEP file" section, and why they behave differently -- Load Session replaces the whole document, so the loaded file's own `/` simply becomes the session's `/` directly, nothing to reconcile. Import STEP keeps the current session intact and nests the new content under its existing `/`, first unwrapping any `/` the incoming file carries of its own (recursively, however many levels deep) -- exactly what lets this file, or any other KodaCAD-produced STEP file, import cleanly without ever ending up with `/` nested inside `/`.

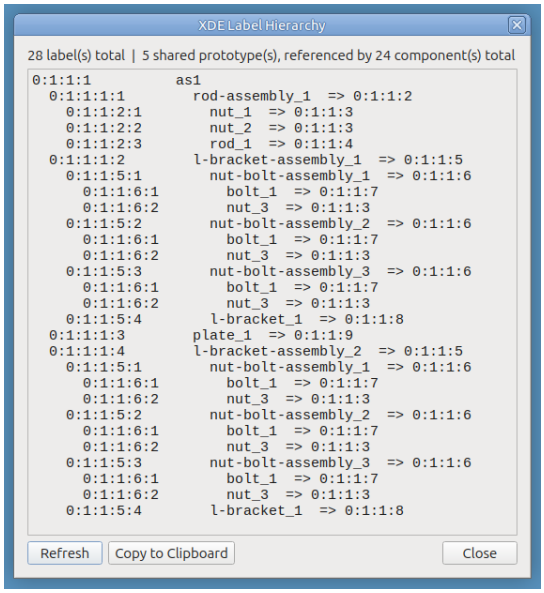
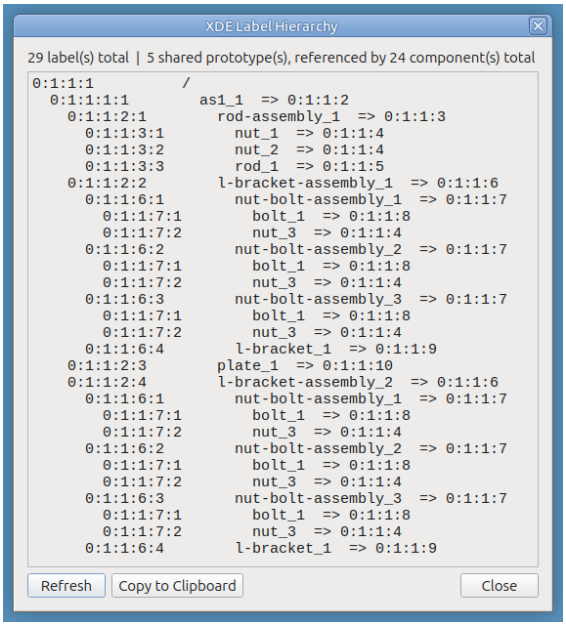
Step 2 -- Explore the structure

Utility -> XDE Label Hierarchy... Walk the tree that opens.

Note which labels are components (occurrences, referring to a prototype) versus the prototypes themselves.

Format: component name-of-component => prototype

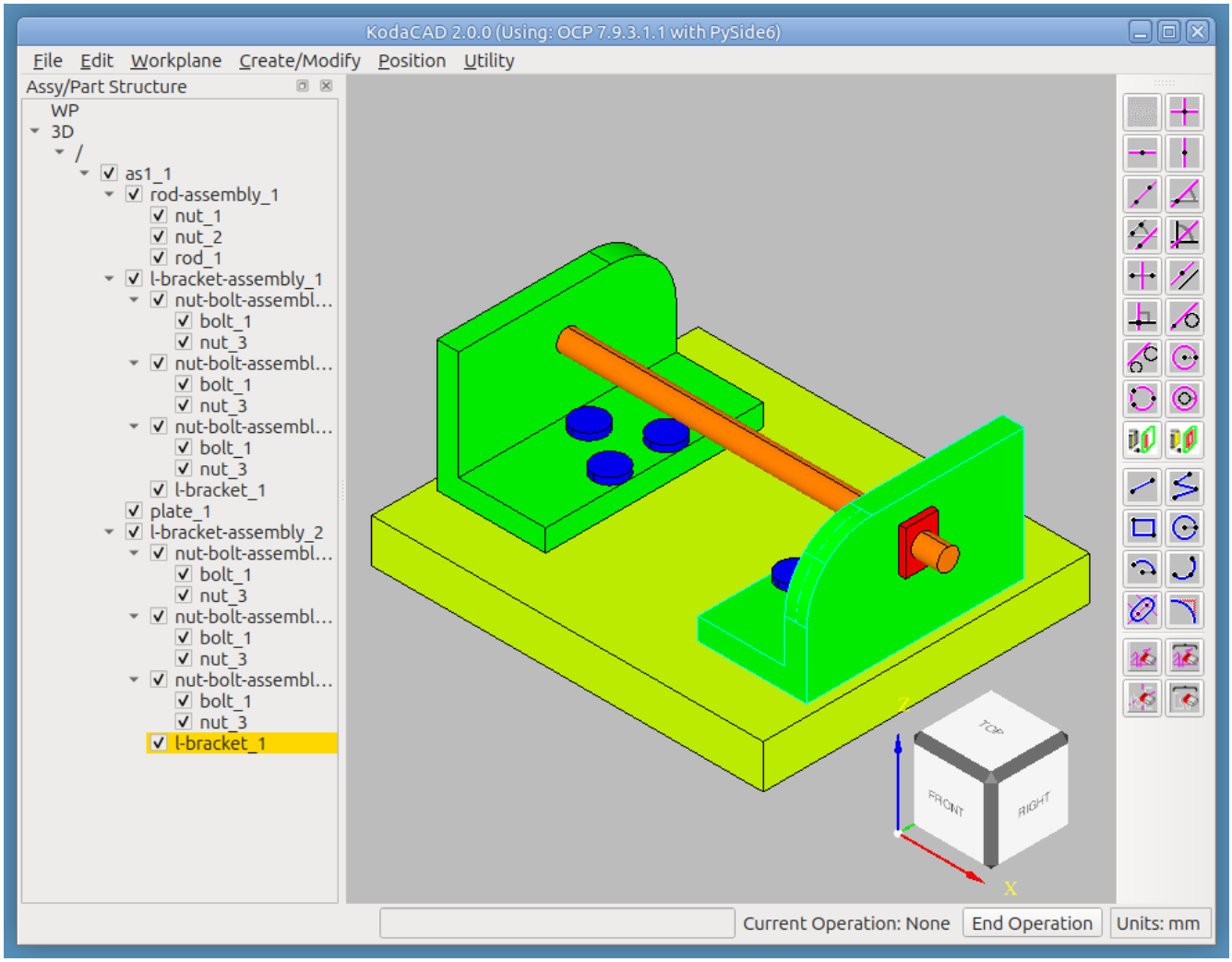
Load file `as1-oc-214.stp` as session	Import file `as1-oc-214.stp` into empty session

Load file `as1-oc-214.stp` as session	Import file `as1-oc-214.stp` into empty session
	

Exercises: XDE hierarchy viewer; recognizing a shared prototype from its referring components.

Step 3 -- Fillet a shared part

Set one of the shared L-bracket instances active, **Create/Modify -> Fillet** on one of its corners. Confirm *both* L-brackets (every instance sharing that prototype) show the fillet, not just the one you picked.



Exercises: fillet's propagation to every instance sharing a prototype -- a prior bug left sibling instances' displays stale even though the underlying shared geometry was already correct. Doing this on `as1-oc-214.stp` specifically (rather than a freshly-created shared instance, as in the Chassis tutorial) confirms the fix also holds for sharing relationships that came in from an imported file rather than being built in the current session.

Step 4 -- See through the plate

RMB click `plate` in the tree and select **Set Transparent**. Confirm the plate becomes see-through in the viewport, revealing the bolt heads and L-bracket geometry that would otherwise be hidden beneath it. RMB click `plate` again and select **Set Opaque** to restore it.

Exercises: Set Transparent / Set Opaque from the tree's own RMB menu.

Step 5 -- Delete one of the shared L-bracket assemblies then Undo

RMB click `l-bracket-assembly_2` (or whichever of the two shared instances you didn't fillet in Step 3) and select **Delete**. Confirm:

- The deleted instance disappears from both the tree and the viewport.

- The *other* L-bracket assembly is completely unaffected -- still present, still filleted, still correctly showing every nut, bolt, and bracket it had before.
- **Undo** this delete. The deleted assembly is restored.

Exercises: deleting one occurrence of a shared prototype while the other survives -- the underlying prototype must not be torn down just because one of its two occurrences was removed, since the remaining occurrence still refers to it. Also exercises Undo.

Step 6 -- Mill a hole into the plate

- **Workplane -> On Face**, click the plate's top face, then a side face for the +U direction.
- Sketch a circle somewhere on the plate that doesn't intersect any existing hole.
- Set the plate active, then **Create/Modify -> Pull**.
- In the dialog:
  - Operation: **Remove Material** (Mill) -- Direction defaults to **-W** automatically once Remove Material is selected (milling straight down into the part), unless you touch the Direction field yourself.
  - Distance (mm): enter something less than the plate's own thickness.
  - Click ✅ **Done**.
- Confirm the hole appears in the viewport, cut straight down into the plate.

Exercises: Pull with Remove Material operation.

Notes for whoever runs this next

- If any menu label, dialog control name, or button text has drifted from what's written here, that's worth fixing in this document directly -- catching that drift is exactly what this tutorial is for.
- See the Quaoar Chassis tutorial (../chassis/README.md) for the broader assembly-construction workflow: creating sibling assemblies, reparenting, creating and positioning a shared instance, and undo/redo across a mixed chain of position- and shape-changing operations.
- See Build Assembly `as1` from a Kit of Parts (../build-as1-oc-214/README.md) for the reverse of this tutorial: starting from `as1-oc-214` 's five loose prototype parts and rebuilding the real assembly from scratch, exercising nearly every Position dialog method along the way.
- See the Lathe tutorial (../lathe/README.md) for organizing and navigating a large, messily-imported assembly -- renaming, sibling assemblies, and reparenting an assembly that already has a parent to a different one.